

SET-3 — MASTER PRACTICAL GUIDE

Online Recruitment System

Windows PC • Eclipse • Git/GitHub • Maven • Tomcat • Docker

Repository	https://github.com/ssvkotamraju/ORS.git
Coverage	28 individual task entries, each followed immediately by an answer
Exam strategy	Command-first, GUI-supported, with older-version fallbacks and examiner-proof checks

Use this as the exam-day walkthrough: open Git Bash/Command Prompt, Eclipse, Docker Desktop, and a browser side by side. For every task, perform the command, verify the result, and only then move on.

HOW TO USE THIS MASTER PDF

This guide is written for a Windows PC and is designed for practical-exam demonstration. Commands are the most version-tolerant method; GUI routes are included where Eclipse/Docker Desktop can be used. Exact menu labels can differ slightly between old and new releases, so use the command equivalent if a GUI label is missing.

Important source limitation: several exam questions refer to a “provided pom.xml”, but the uploaded question papers contain the questions, not the actual POM file. For those questions, this guide deliberately shows how to derive the exact value from the supplied POM instead of inventing coordinates, WAR names, or context paths.

1. Windows tool setup

Tool	What to verify	Windows command / GUI
Java JDK	Use a JDK, not only a JRE.	<code>java -version</code> <code>javac -version</code>
Maven	Maven available to the terminal, or Maven Wrapper exists.	<code>mvn -version</code> <code>mvnw.cmd -version</code>
Git	Git Bash/Command Prompt can run Git.	<code>git --version</code>
Eclipse	Maven integration (m2e) and Java support.	Window → Preferences → Java
Tomcat	Choose a Tomcat version compatible with the project.	Servers view → New → Apache Tomcat
Docker	Docker daemon/Desktop is running.	<code>docker version</code> Docker Desktop GUI

If `mvn` is not recognized

Either install/configure Maven and add its **bin** directory to Windows PATH, or check whether the repository contains **mvnw.cmd**. The wrapper is often the safest option for older student projects because it uses the project's expected Maven distribution.

```
where mvn
mvn -version
mvnw.cmd -version
```

Windows GUI: Control Panel / Settings → System → Advanced system settings → Environment Variables → create MAVEN_HOME=C:\apache-maven-3.x.x → edit Path → add %MAVEN_HOME%\bin → reopen terminal.

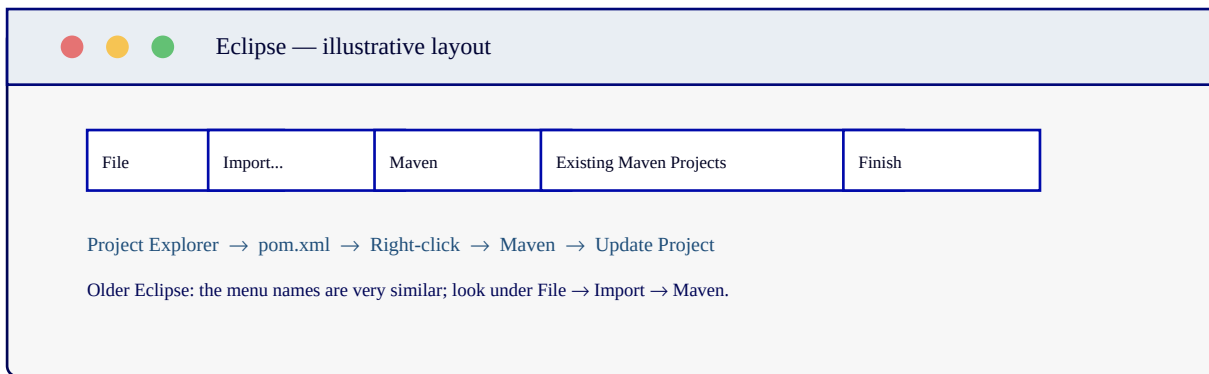
Java compatibility rule

Always run **mvn -version**, not only **java -version**, because Maven reports the Java home it actually uses. If an old project expects Java 8/11 and a newer JDK is installed, align the JDK used by Maven and Eclipse with the project's compiler settings. Do not blindly change the POM until you know the required source/target/release level.

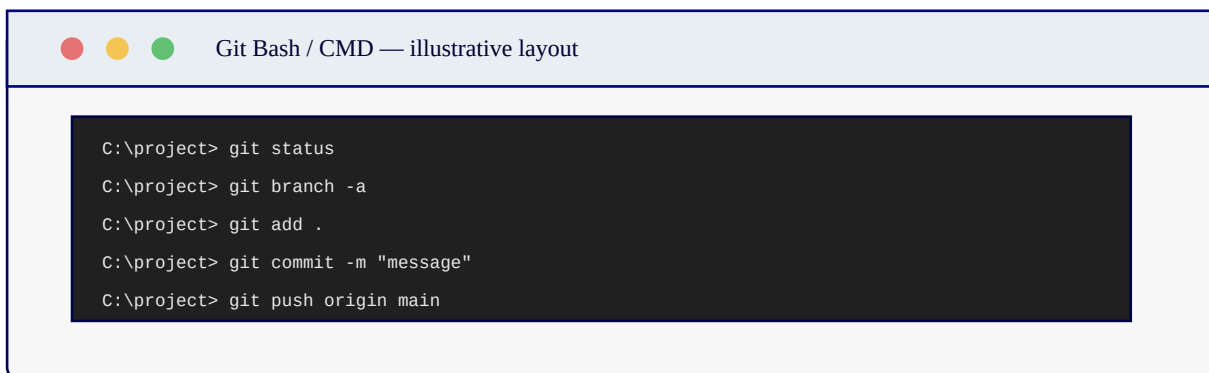
Tomcat compatibility warning

Older JSP/Servlet projects commonly use the **javax.servlet** API. Tomcat 10+ moved to the **jakarta.servlet** namespace. If the project imports `javax.servlet.*`, Tomcat 9 (or another compatible javax-based server) is usually the safer exam choice unless the code has already been migrated to Jakarta.

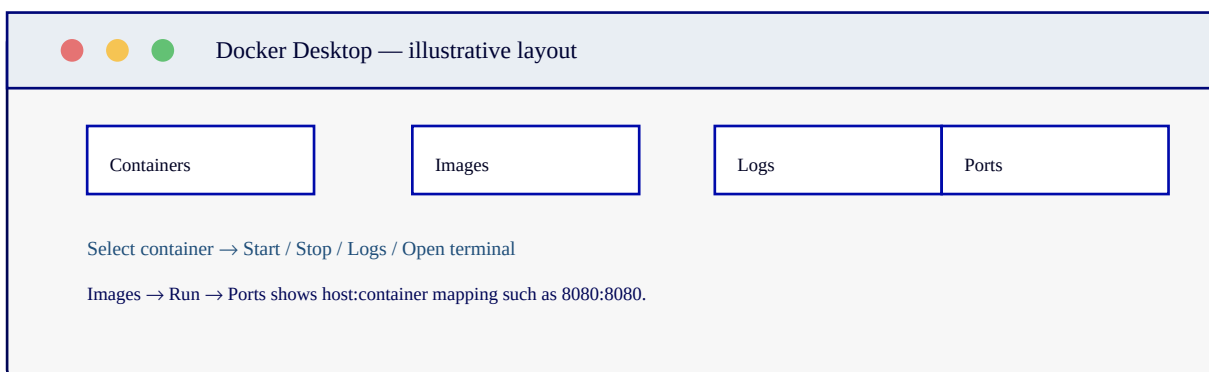
2. GUI routes you can demonstrate to the examiner



Eclipse Maven import: **File** → **Import** → **Maven** → **Existing Maven Projects** → Browse to the cloned folder → select pom.xml → Finish. Then right-click the project → Maven → Update Project. For older Eclipse releases, the wording is almost identical.



Git tasks are best demonstrated in Git Bash because every advanced command in these papers is available there. GitHub Desktop/Eclipse EGit can perform many common operations, but rebase, reflog, patch, and detailed recovery are much easier to prove from Git Bash.



Docker Desktop is useful for showing Images, Containers, Logs, and port mappings. If the lab machine has an older Docker Desktop layout, the same objects may appear under older tabs/menu names. The CLI remains the most reliable fallback.

3. Universal exam command sheet

Area	Commands
Git basics	<code>git clone URL</code> <code>cd project</code> <code>git status</code> <code>git remote -v</code> <code>git branch -a</code>
Branching	<code>git switch -c feature/name</code> # older Git: <code>git checkout -b feature/name</code>
Inspect changes	<code>git diff</code> <code>git diff --cached</code> <code>git status</code>
Commit	<code>git add file</code> <code>git commit -m "message"</code> <code>git log -1 --oneline</code>
Remote sync	<code>git fetch origin</code> <code>git merge origin/main</code> <code>git pull origin main</code>
Rebase	<code>git fetch origin</code> <code>git rebase origin/main</code> <code>git rebase --abort</code>
Undo safely	<code>git revert <hash></code> <code>git reset --soft HEAD~1</code> <code>git reflog</code>
Stash	<code>git stash push -m "work"</code> <code>git stash list</code> <code>git stash pop</code>
Compare/history	<code>git diff main feature -- file</code> <code>git log --oneline --graph --decorate --all</code>
Maven	<code>mvn -version</code> <code>mvn clean package</code> <code>mvn dependency:tree</code> <code>mvn dependency:analyze</code> <code>mvn clean package -X</code>
Tests	<code>mvn test</code> <code>mvn -Dtest=ClassName test</code> <code>mvn clean package -DskipTests</code>
Docker	<code>docker build -t app:v1 .</code> <code>docker run -d --name app -p 8080:8080 app:v1</code> <code>docker ps</code> <code>docker logs app</code>
Docker Hub	<code>docker login</code> <code>docker tag app:v1 user/app:v1</code> <code>docker push user/app:v1</code> <code>docker logout</code>

4. Universal troubleshooting decision tree

Maven failure: read the first meaningful [ERROR] → identify the phase/goal → inspect the POM coordinates/configuration → run `dependency:tree/effective-pom` → check JDK with `mvn -version` → clean → rebuild with `-X`.

Browser failure: prove Maven created the artifact → prove the server started → prove the port is listening → prove the context path → read server logs → then inspect JSP/application code.

Docker failure: prove Docker daemon → prove Dockerfile/context → prove image exists → prove container exists → inspect `docker ps -a` → inspect docker logs → inspect port mapping.

5. COMPLETE TASK-BY-TASK QUESTION → ANSWER GUIDE

The task wording below is kept close to the uploaded question paper while being shortened where a long scenario is repeated. Every numbered/sub-numbered practical requirement is represented. Where the source asks for an exact value that depends on an unseen POM, the answer explicitly tells you how to derive it.

Q0 — TASK / QUESTION

Online Recruitment System scenario: summarize the main functional and non-functional requirements implied by the problem.

ANSWER / PROCEDURE

Translate the scenario into functional and non-functional requirements, actors, core use cases, and deployment needs. For a web hospital/ticketing/recruitment system, include authentication, role-based access, CRUD workflows, reports, security, usability, and concurrent-user performance.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

1(a) — TASK / QUESTION

Clone ORS, import it into Eclipse, and verify that pom.xml is recognized.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/ssvkotamraju/ORS.git
cd ORS
git status
git remote -v
```

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(b) — TASK / QUESTION

Identify problematic dependencies, modify pom.xml, update Maven, and verify the build.

ANSWER / PROCEDURE

In Eclipse, right-click the project → Maven → Update Project... (older Eclipse/m2e versions use the same Maven menu). Check Force Update of Snapshots/Releases when a stale dependency is suspected, then Project → Clean.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(c) — TASK / QUESTION

Identify a Maven dependency conflict and select a compatible version.

ANSWER / PROCEDURE

Use the dependency tree to see which paths bring different versions. Maven normally selects the nearest definition in the dependency graph; a dependencyManagement entry or explicit direct dependency can be used to pin the intended version. Exclusions remove an unwanted transitive path.

Commands / configuration

```
mvn dependency:tree -Dverbose
mvn help:effective-pom
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(d) — TASK / QUESTION

Resolve a Java-version compatibility issue in pom.xml and Eclipse.

ANSWER / PROCEDURE

Set the workspace JDK under Window → Preferences → Java → Installed JREs; select the compatible JDK. Then Project → Properties → Java Compiler and Maven → Update Project. For old Eclipse, install a JDK it supports; a very new JDK may not work with an old Eclipse release.

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(e) — TASK / QUESTION

Troubleshoot, clean the project, refresh dependencies, and confirm a successful build.

ANSWER / PROCEDURE

In Eclipse, right-click the project → Maven → Update Project... (older Eclipse/m2e versions use the same Maven menu). Check Force Update of Snapshots/Releases when a stale dependency is suspected, then Project → Clean.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

1(f) — TASK / QUESTION

Diagnose server/port settings so ORS can be accessed in a browser.

ANSWER / PROCEDURE

A successful Maven build only proves the build lifecycle completed. Verify the WAR/JAR exists, start the server, confirm the server log says it started, confirm the expected port is listening, then open the correct context path. Check browser/server logs for 404/500 errors.

Commands / configuration

```
dir target
netstat -ano | findstr :8080
tasklist | findstr /I java
```

GUI route: Eclipse → Window → Show View → Servers → add/start Tomcat; then open the URL in a browser and check the Servers/Console view for startup errors.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

1(g) — TASK / QUESTION

Find the process using port 8080 and move ORS to another port.

ANSWER / PROCEDURE

On Windows, identify the process listening on port 8080, map the PID to the process, then change the application/server port to a free port such as 8081. In Docker, use host:container mapping.

Commands / configuration

```
netstat -ano | findstr :8080
tasklist /FI "PID eq <PID>"
docker ps
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

1(h) — TASK / QUESTION

Find the Maven compiler configuration and set a Java version supported by the installed JDK.

ANSWER / PROCEDURE

Check `<maven.compiler.source>`/`<maven.compiler.target>` or the `maven-compiler-plugin` configuration. Use a compiler level supported by the installed JDK and by the codebase. In older projects you will often see `1.8`; modern Maven Compiler Plugin versions also support a `release` setting.

Commands / configuration

```
mvn -version
mvn help:effective-pom
mvn clean compile
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(i) — TASK / QUESTION

Determine whether a dependency problem comes from the declaration, repository, network, or Maven cache.

ANSWER / PROCEDURE

Run Maven with debug logging and preserve the first meaningful error. Do not start by changing random POM values. Check the first `[ERROR]`, the goal/phase named, and the `Caused by:` chain.

Commands / configuration

```
mvn clean package -X
mvn clean package -e
mvn -version
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

1(j) — TASK / QUESTION

Compare JDK, Maven version, local repository, and project dependencies across systems.

ANSWER / PROCEDURE

First compare the JDK Maven actually uses with the project compiler level. `mvn -version` is better than only `java -version` because it reports the Java home used by Maven. Then align Eclipse's installed JRE/JDK and the POM compiler settings.

Commands / configuration

```
java -version
javac -version
mvn -version
where java
where mvn
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

2(a) — TASK / QUESTION

Clone ORS, create a working branch, and verify the remote connection.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/ssvkotamraju/ORS.git
cd ORS
git status
git remote -v
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(b) — TASK / QUESTION

Retrieve latest remote changes and determine whether the branch is behind.

ANSWER / PROCEDURE

Fetch downloads new remote objects and references but does not merge them into your current working tree. This is the safe answer when the question says 'retrieve without modifying local files'.

Commands / configuration

```
git fetch origin
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(c) — TASK / QUESTION

Update local information and rebase the current work onto the latest remote branch.

ANSWER / PROCEDURE

Fetch first, then rebase your feature branch onto the updated main branch. This rewrites the feature branch commit IDs, so do it before sharing the branch or be prepared to use `--force-with-lease` when pushing.

Commands / configuration

```
git fetch origin
git switch feature/your-branch
git rebase origin/main
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(d) — TASK / QUESTION

Resolve a rebase conflict, stage the file, and continue.

ANSWER / PROCEDURE

When Git stops on a conflict, open the file, find `<<<<<<<`, `====`, `>>>>>>>`, choose/edit the correct content, save, stage, and continue. Repeat until `git rebase --continue` finishes.

Commands / configuration

```
git status
git diff
git add <resolved-file>
git rebase --continue
git status
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(e) — TASK / QUESTION

Resolve all rebase conflicts systematically and verify completion.

ANSWER / PROCEDURE

When Git stops on a conflict, open the file, find `<<<<<<<`, `====`, `>>>>>>>`, choose/edit the correct content, save, stage, and continue. Repeat until `git rebase --continue` finishes.

Commands / configuration

```
git status
git diff
git add <resolved-file>
git rebase --continue
git status
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(f) — TASK / QUESTION

Investigate history and restore missing changes without losing your own work.

ANSWER / PROCEDURE

``reflog`` records where HEAD and branch references pointed recently, even after a reset/rebase. Find the lost commit, inspect it, then recover it with a new branch or cherry-pick.

Commands / configuration

```
git reflog
git show <lost-hash>
git branch recovered-work <lost-hash>
git cherry-pick <lost-hash>
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

2(g) — TASK / QUESTION

Cancel the rebase and return to the previous state.

ANSWER / PROCEDURE

If a rebase is in progress and you want to abandon it completely, abort it. The branch is returned to the state it had before the rebase started.

Commands / configuration

```
git rebase --abort
```

Examiner proof: run ``git status`` and/or ``git log -1 --oneline``; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(h) — TASK / QUESTION

Configure an SSH key, add it to GitHub, test it, and retry the push.

ANSWER / PROCEDURE

Generate an SSH key, start the Windows OpenSSH agent if necessary, add the key, copy the ``.pub`` contents into GitHub → Settings → SSH and GPG keys, then test the connection. Use the repository's SSH URL as the remote. On older Windows/Git versions, Git Bash is usually the easiest route.

Commands / configuration

```
ssh-keygen -t ed25519 -C "your_email@example.com"
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
ssh -T git@github.com
git remote -v
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run ``git status`` and/or ``git log -1 --oneline``; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(i) — TASK / QUESTION

Verify the correct `apply.jsp` on GitHub and confirm only intended changes were committed.

ANSWER / PROCEDURE

Push the branch and then compare the local commit ID with the remote-tracking branch after fetching. Equal hashes mean both point to the same commit.

Commands / configuration

```
git push -u origin main
git fetch origin
git rev-parse HEAD
git rev-parse origin/main
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run ``git status`` and/or ``git log -1 --oneline``; show the relevant branch/remote/diff output. Do not only say the command — show its result.

2(j) — TASK / QUESTION

Choose the appropriate push after a rebase and explain precautions.

ANSWER / PROCEDURE

After a rebase, the rewritten branch usually needs a force push. Prefer ``--force-with-lease`` because it refuses to overwrite unexpected remote work. Never casually force-push a shared main branch.

Commands / configuration

```
git fetch origin
git push --force-with-lease origin feature/your-branch
```

Examiner proof: run ``git status`` and/or ``git log -1 --oneline``; show the relevant branch/remote/diff output. Do not only say the command — show its result.

3(i) — TASK / QUESTION

Create a Dockerfile and build an ORS image for Docker Hub.

ANSWER / PROCEDURE

For a JSP/Servlet WAR, use Tomcat as the runtime. The WAR is copied into Tomcat's ``webapps`` directory. A multi-stage build is cleaner because Maven is used only in the build stage.

Commands / configuration

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9-jdk17
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY --from=build /app/target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show ``docker images`` or ``docker ps -a``, then show logs/ports when relevant. For browser tasks, actually open ``http://localhost:8080``.

3(ii) — TASK / QUESTION

Run ORS with container port 8080 mapped to host port 8080.

ANSWER / PROCEDURE

Start detached, map host 8080 to container 8080, and name the container so it is easy to demonstrate.

Commands / configuration

```
docker run -d --name ors-container -p 8080:8080 ORS:v1
docker ps
docker port ors-container
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show ``docker images`` or ``docker ps -a``, then show logs/ports when relevant. For browser tasks, actually open ``http://localhost:8080``.

3(a) — TASK / QUESTION

What information is needed before creating the ORS Dockerfile and what major instructions are required?

ANSWER / PROCEDURE

Confirm ``pom.xml``, ``src/main/webapp``, the JSP files, Java version, WAR artifact name, Tomcat version, and port. Build the WAR first, then containerize it.

Commands / configuration

```
dir pom.xml
dir src\main\webapp
mvn clean package
dir target
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(b) — TASK / QUESTION

Troubleshoot the Dockerfile, project structure, artifact, and build context.

ANSWER / PROCEDURE

Read the build error from the first failing instruction, check the build context and file names, confirm the Dockerfile is in the current directory, and rebuild without guessing. For runtime failures, inspect `docker ps -a` and `docker logs`.

Commands / configuration

```
docker build -t app:v1 .
docker ps -a
docker logs <container>
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(c) — TASK / QUESTION

Map container port 8080 to host port 8080.

ANSWER / PROCEDURE

A container port is not automatically published to the Windows host. `-p 8080:8080` means HOST_PORT:CONTAINER_PORT. Verify the mapping with `docker port` and then browse to localhost:8080.

Commands / configuration

```
docker run -d --name app -p 8080:8080 app:v1
docker port app
netstat -ano | findstr :8080
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(d) — TASK / QUESTION

Reduce image size and improve Docker build efficiency.

ANSWER / PROCEDURE

Use a multi-stage build, copy only files needed for the build, use a `.dockerignore`, choose a smaller runtime image, and avoid leaving Maven caches in the runtime image.

Commands / configuration

```
type nul > .dockerignore
docker build -t app:v1 .
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

3(e) — TASK / QUESTION

Tag, authenticate, push, and verify the ORS image on Docker Hub.

ANSWER / PROCEDURE

Authenticate, tag the local image with your Docker Hub namespace, push it, then verify on Docker Hub or with `docker pull` from another machine.

Commands / configuration

```
docker login
docker tag ORS:v1 <dockerhub-username>/ORS:v1
docker push <dockerhub-username>/ORS:v1
docker pull <dockerhub-username>/ORS:v1
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

6. EXAMINER DEMONSTRATION CHECKLIST

- Open the project folder and prove the repository/remote is correct.
- Open pom.xml and point to groupId, artifactId, version, packaging, dependencies and plugins when asked.
- Run mvn -version and, if required, java -version/javac -version.
- Build with mvn clean package (or the exact lifecycle command in the question).
- Show target/ and the generated WAR/JAR.
- In Eclipse, use Maven → Update Project when dependencies are stale.
- For Git, show status/diff before commit and log after commit.
- For rebase/merge questions, show the branch name and final clean status.
- For Docker, show docker images, docker ps -a, docker logs and port mapping as appropriate.
- Open http://localhost:8080 only after proving the server/container is running.
- For Docker Hub, show tag → login → push → repository verification.

7. OLD-VERSION FALLBACKS

If you see...	Use this fallback
Old Git without `git switch`	Use `git checkout -b branch` / `git checkout branch`.
Old Eclipse	Look under File → Import → Maven and Window → Show View → Servers; labels vary slightly.
Old Maven project	Check for `mvnw.cmd`; otherwise use a Maven version compatible with the project/JDK.
Old Tomcat project using javax.servlet	Prefer a javax-compatible Tomcat release (commonly Tomcat 9) unless the project is migrated to Jakarta.
Docker Desktop GUI differs	Use the CLI; the core objects remain images, containers, logs, and port mappings.
PowerShell command differs	Git Bash is the simplest universal shell for the Git commands in these papers.

Official references used for the practical workflow

Git for Windows: <https://git-scm.com/install/windows>

Eclipse documentation: <https://www.eclipse.org/documentation/>

Maven Compiler Plugin: <https://maven.apache.org/plugins/maven-compiler-plugin/>

Maven Dependency Plugin: <https://maven.apache.org/plugins/maven-dependency-plugin/>

Docker Desktop for Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>